

Concept Proposal

Viola's Kitchen: A Single-Admin Cooking Blog

Portfolio, Phase 1 (Conception Phase)

Course: Project: Getting Started in Web Programming (DLBITPEWP01_E)

Task 3: Blog/Content Platform

Author: Violeta Driza

Matriculation Number: IU14102196

Tutor: Andrew Adjah Sai

Date: 11th September 2026

1. Purpose and Concept

Viola's Kitchen is a single-admin cooking blog developed for the course Project: Getting Started in Web Programming (DLBITPEWP01_E), Task 3: Blog/Content Platform. As the site's sole author, I publish original recipes, and visitors browse and discuss them. The platform demonstrates a full-stack blog/content management system: server-rendered pages, a relational database of posts (recipes) and comments, and a small administrative workflow for content management. This realizes the core CRUD (create, read, update, delete) pattern required by the task, applied to a themed content type (recipes) rather than generic blog posts.

2. Features

- Recipe listing: a home page listing recipes, filterable by one of five categories: Air Fryer, Desserts, Breakfast, Instant Pot, and Dinner.
- Recipe detail page: a full recipe view comprising title, ingredients, instructions, preparation time, image, author, and publication date.
- Comments: visitors may leave a name and comment beneath any recipe. Submission is handled asynchronously via the Fetch API, so the comment is appended without a page reload.
- Admin authoring: a login-protected page through which I add new recipes via a form. No public registration is offered.
- No visitor accounts: visitor registration is intentionally omitted to keep the scope realistic for the course timeline, in line with the single-author model suggested by the assignment.

3. User Roles and Authentication

Two roles are defined: a single hard-coded administrator account, held by me, and anonymous visitors. The administrator authenticates through a password-protected /admin/login route; the password is stored as a bcrypt hash rather than in plain text, and the resulting session is tracked using express-session. Visitors require no account to read recipes or submit comments. This design keeps the project's scope realistic while still demonstrating authentication and session handling, as permitted by the assignment brief.

4. Data Model

Two entities represent the site's content:

- recipes: id, title, category, ingredients, instructions, prep_time, image_url, created_at, author.
- comments: id, recipe_id (foreign key referencing recipes.id), author_name, comment_text, created_at.

Each recipe may have many comments, forming a one-to-many relationship. No separate users table is maintained beyond the single administrator credential, which is kept in environment configuration rather than the database. This relationship is illustrated in the diagram in Section 6.

5. Technology Stack

Backend: Node.js with Express.js for routing and middleware.

Database: MySQL, accessed through the mysql2 driver.

Frontend: Server-rendered HTML via EJS templates, styled with hand-written CSS3; vanilla JavaScript with the Fetch API drives the asynchronous comment form.

Authentication/Session: express-session for login state; bcrypt for password hashing.

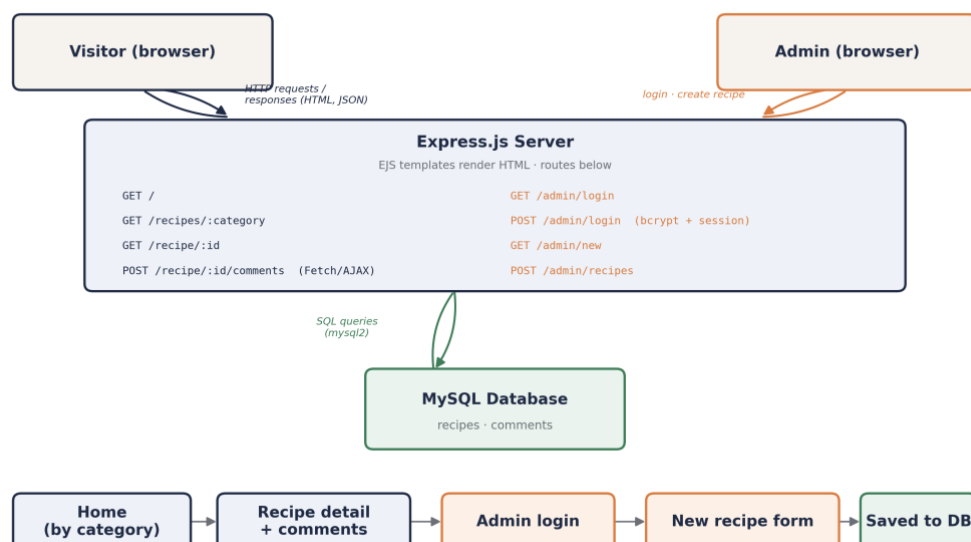
Tooling: dotenv for environment configuration, nodemon for local development, and Git/GitHub for version control.

Server-side rendering, rather than a client-side framework, keeps the project close to the course's core HTML/CSS/JavaScript/server-language/database stack, while the Fetch-based comment form still demonstrates AJAX and DOM scripting, as encouraged in the task description.

6. Diagram

The diagram below illustrates both the system architecture (visitor and administrator browsers, the Express server and its routes, and the MySQL database) and the page flow a visitor, or administrator follows through the site.

Viola's Kitchen: Architecture & Page Flow



7. Repository

Source code and this concept document are version-controlled and available at: github.com/violetadriza1/violas-kitchen. The repository already contains the initial project scaffold (the Express entry point, folder structure, and dependencies) together with this plan.